Actividad en clase semana 4

Registro de consultas realizadas durante la conversación

N.º	Consulta realizada	Objetivo	Información y palabras clave obtenidas	Cómo llevó a la siguiente
1	Ayúdame a completar esto con esos archivos (captura de carpetas y Herramientas.zip), pero antes dime si necesitas un archivo de mi proyecto.	Completar la estructura controller, model, repository y service según el proyecto real, verificando antes si faltaba algún archivo.	No hacía falta otro archivo: el zip ya traía un CRUD de Product. Palabras clave: Kotlin, Spring Boot, PostgreSQL, Flyway, CRUD, controller, model, repository, service.	Como el proyecto ya tenía código, surgió la duda de qué había que completar exactamente.
2	controller, product, product repository, product service	Indicar cuáles archivos se debían completar.	Se entregaron los cuatro archivos (Product, ProductRepository, ProductService, ProductController) con JpaRepository, búsqueda, validaciones y rutas /api/products. Se notaron detalles del application.yaml.	El usuario mostró su proyecto real en el IDE, que seguía otro estilo.
3	Así tiene que ser según el profe, y por qué el ícono azul cambia a esa banderita morada (captura del IDE).	Ajustar el servicio al estilo del profesor y entender el cambio de ícono.	Ícono morado: el nombre del archivo no coincide con el de la clase. Error rojo por llaves faltantes. Estilo: @Service, open class, @Autowired open lateinit var, open fun. Package en minúscula.	La respuesta incluyó más métodos de los que mostraba la captura y el usuario pidió limitarse.
4	Pero por qué agregas todo eso, solo hazle hasta donde te mandé la captura.	Limitar la respuesta al alcance de la captura.	ProductService solo con listar(), más las llaves corregidas de la clase y la función.	Con el servicio listo, siguió el controller.

N.º	Consulta realizada	Objetivo	Información y palabras clave obtenidas	Cómo llevó a la siguiente
5	Este arregla (código del controller).	Corregir los errores del controller.	open y private no van juntos; faltaban las comillas en "/listar"; @Controller era redundante con @RestController; import sobrante. Palabras clave: @RestController, @GetMapping, @Autowired.	Faltaba el repositorio, que el servicio necesita para funcionar.
6	Qué pongo en ProductRepository (clase vacía).	Saber qué debe contener el repositorio.	Debe ser una interface que extienda JpaRepository<Product, Int>, con @Repository. Ya trae findAll y save.	Con las tres capas listas, el usuario pasó a la base de datos.
7	Hice este V3 (captura de V3__idcorrection.sql y consola).	Validar la migración de Flyway creada por el usuario.	CREATE SEQUENCE, nextval y setval; falta un punto y coma final; no editar una migración ya aplicada (checksum). La consola mostró Tomcat iniciado en el puerto 8081.	Con la app corriendo, preguntó cómo probarla.
8	Cómo pruebo en Postman.	Probar el endpoint existente.	GET http://localhost:8081/listar, respuesta 200 OK y [] si la tabla está vacía. Solo existía el GET.	Pidió agregar el POST.
9	Ayúdame a hacer un post.	Crear productos desde la API.	crear() en el service, @PostMapping("/crear"), @RequestBody, id = null y Body raw en formato JSON.	Quedó la duda de qué archivo había que cambiar.
10	Qué debo cambiar, el service o cuál.	Aclarar qué archivos modificar.	Se cambian el service y el controller, no el repository. Flujo: Postman, controller, service, repository, base de datos.	Con el POST funcionando, pidió PUT y DELETE.

N.º	Consulta realizada	Objetivo	Información y palabras clave obtenidas	Cómo llevó a la siguiente
11	Listo, ya tengo todo eso, ayúdame a hacer el put, delete (con su controller).	Agregar actualizar y eliminar.	actualizar(id, datos) y eliminar(id); @PutMapping, @DeleteMapping y @PathVariable. Pruebas en Postman: PUT /actualizar/1 y DELETE /eliminar/1.	El usuario quiso entender la sintaxis para aplicarla por su cuenta.
12	Explícame la sintaxis para yo mismo aplicarlo al archivo.	Entender el patrón para repetirlo sin ayuda.	Patrón controller, service, repository. Conceptos: @PathVariable, @RequestBody, Optional, orElse, operador elvis, save() hace UPDATE o INSERT según el id, existsById y deleteById.	Con el proceso entendido, pidió documentarlo en esta tabla.